

Python Assignment: Build a Hangman Game

Objective

Create a simple **Hangman Game in Python** where the computer chooses a secret word and the player has to guess the word one letter at a time.

The player should have a limited number of wrong attempts. Every time the player guesses an incorrect letter, they lose one chance.

You will build the game step by step using basic Python concepts.

Concepts to Practice

- Variables
- Strings
- `input()`
- `print()`
- Type conversion where required
- `if`
- `elif`
- `else`
- `while` loops
- `for` loops
- Comparison operators
- Logical operators
- String indexing
- Counters
- Basic string operations

Do not use solutions or advanced Python concepts that have not been taught yet.

Part 1 — Create the Basic Game

Create the basic structure of the Hangman game.

Requirements

1. Create a secret word.
2. Display a welcome message.
3. Ask the player to guess a letter.
4. Check whether the guessed letter exists in the secret word.
5. Give appropriate feedback.

Example

Welcome to Hangman!

Guess a letter: a

Good guess! The letter 'a' is in the word.

If the letter is not present:

Guess a letter: z

Wrong guess! The letter 'z' is not in the word.

Choose your own secret word.

Part 2 — Display the Hidden Word

The player should not see the secret word immediately.

Instead, display a placeholder for every letter.

For example, if the secret word is:

python

The player should initially see:

Requirements

- One underscore should represent each letter.
- The actual letters should remain hidden.
- The player should be able to see how many letters the word contains.

Example

The word has 6 letters.

Think about how you can use a loop to display the placeholders.

Part 3 — Reveal Correct Letters

Now make the game more interactive.

If the player correctly guesses a letter, reveal that letter in its correct position.

For example:

Secret word

Player guesses

p

Display

p _ _ _ _

If the player then guesses:

o

Display:

p _ _ o _ _

Continue revealing correctly guessed letters.

Requirements

- Correct letters should appear in their correct positions.
 - Letters that have not been guessed should remain hidden.
 - The secret word itself should not be displayed.
-

Part 4 — Handle Incorrect Guesses

If the player enters a letter that does not exist in the secret word, the player should lose one chance.

Example

Guess a letter: z

Wrong guess!

Chances remaining: 5

Requirements

Create a variable to keep track of the player's remaining chances.

For example:

chances = 6

You can choose the number yourself.

- Every incorrect guess should reduce the number of chances.
 - A correct guess should not reduce the chances.
-

Part 5 — Create the Main Game Loop

The game should continue asking the player for letters.

Use a `while` loop to control the game.

The loop should continue while:

- The player has not guessed the complete word.
- The player still has chances remaining.

The game should stop when either:

Condition 1:

The player successfully guesses the word.

OR

Condition 2:

The player runs out of chances.

Think carefully about the condition for your `while` loop.

Part 6 — Check Whether the Player Has Won

After every correct guess, check whether the player has discovered all the letters.

Example

`p y t h _ _`

The player guesses:

`o`

Then:

`p y t h o _`

If the player guesses:

`n`

Then:

`p y t h o n`

Winning Example

`p y t h o n`

Congratulations!

You guessed the word!

Requirements

- Check the winning condition after each guess.
 - Stop the game immediately when the complete word has been revealed.
 - Display a suitable winning message.
-

Part 7 — Prevent Invalid Guesses

Think about what should happen if the player enters something that isn't a single letter.

For example:

Guess a letter: python

or:

Guess a letter: 7

or:

Guess a letter:

Requirements

The program should display an appropriate message such as:

Please enter one letter only.

Then ask the player for another guess.

An invalid input should **not** unnecessarily reduce the player's chances.

Part 8 — Prevent Repeated Guesses

Now handle a common situation.

Suppose the player already guessed:

a

and then enters:

a

again.

The game should recognize that the player has already guessed that letter.

Example

You already guessed 'a'.

Try a different letter.

Requirements

Keep track of the letters the player has already guessed.

If a letter has already been entered:

- Do not count it as a new guess.
 - Do not reduce the player's chances.
 - Ask the player to enter another letter.
-

Part 9 — Display the Guessed Letters

Give the player a clear view of their progress.

Example

Word: p _ t _ _ _

Guessed letters: a, e, p, t, o

Chances remaining: 4

Requirements

After every valid guess, display:

1. The current state of the word.
2. Letters already guessed.
3. Chances remaining.

Keep the display simple and easy to understand.

Part 10 — Add a Hangman Display

Now make the game feel more like an actual Hangman game.

Create different stages based on the number of incorrect guesses.

For example:

Wrong guesses: 0

```
+---+
|   |
|   |
```

```
|
|
|
=====
```

Wrong guesses: 1

```
+---+
|  |
|  O
|  |
|  |
|  |
=====
```

Continue adding parts as the player makes more incorrect guesses.

Requirements

Create several stages.

For example:

1. Empty gallows
2. Head
3. Body
4. One arm
5. Second arm
6. One leg
7. Second leg

You decide exactly what your drawing looks like.

Use `if`, `elif`, or another basic approach you have learned to decide which stage should be displayed.

Part 11 — Create a Game-Over Condition

The player should lose when they use all available chances.

Example

```
+---+
|  |
|  O
| /\
```

```
| /\n|\n=====\nGame Over!
```

You ran out of chances.

The word was: python

Requirements

When the player loses:

- Stop asking for guesses.
 - Display the final Hangman drawing.
 - Display a game-over message.
 - Reveal the secret word.
-

Final Complete Hangman Game

Now combine everything you have built in Parts 1–11.

Your final program should include the following:

Game Start

Welcome to Hangman!

Game Setup

The program should:

1. Create a secret word.
2. Display the hidden word.
3. Set the player's chances.
4. Set up the variables needed to track guesses.

During the Game

The player should:

1. Enter a letter.
2. Have the input checked.
3. Know whether the letter is correct or incorrect.

4. See the updated word.
5. See previously guessed letters.
6. See remaining chances.
7. See the Hangman drawing.
8. Continue until they win or lose.

Winning

If the player guesses the complete word:

Congratulations!

You guessed the word!

Losing

If the player runs out of chances:

Game Over!

You ran out of chances.

The correct word was: python

Suggested Final Game Flow

START



Display welcome message



Create secret word



Create hidden word



Set chances



Start guessing loop



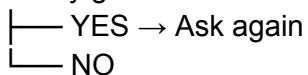
Ask for a letter

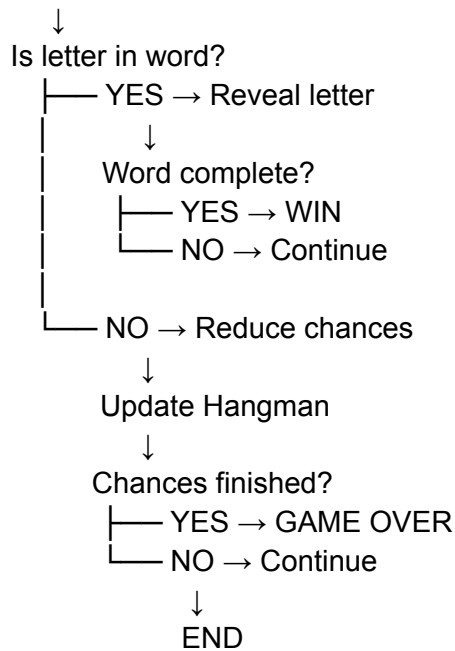


Validate input



Already guessed?





Rules for This Assignment

1. Use `if`, `elif`, and `else`.
 2. Use `while` loops.
 3. Use `for` loops where appropriate.
 4. Use strings and string operations.
 5. Take input from the player.
 6. Use variables and counters.
 7. Keep track of guesses.
 8. Use clear variable names.
 9. Keep your code properly indented.
 10. Add comments for difficult parts.
 11. Do not use classes.
 12. Do not use advanced Python concepts.
 13. Do not copy a complete Hangman program from the internet.
 14. Build the program step by step.
 15. Test each part before moving to the next step.
-

Final Submission Checklist

Before submitting your assignment, make sure:

- The game runs correctly.

- The secret word is hidden.
 - Underscores are displayed for hidden letters.
 - Correct letters are revealed.
 - Incorrect guesses reduce chances.
 - The Hangman drawing changes after wrong guesses.
 - The player cannot enter multiple letters as one guess.
 - Invalid input is handled.
 - Repeated letters are handled.
 - Previously guessed letters are displayed.
 - The player can win.
 - The player can lose.
 - The secret word is revealed after losing.
 - Different possible situations have been tested.
 - The code is properly formatted and readable.
 - No complete Hangman solution was copied.
-

Main Goal

Do not try to build the complete Hangman game in one go.

Build the program step by step.

Start with:

One letter check



Hide the word



Reveal correct letters



Add the guessing loop



Add chances



Handle invalid input



Handle repeated guesses



Display guessed letters



Add the Hangman drawing



Handle winning and losing



Complete Hangman Game

The goal is not just to make Hangman work.

The goal is to understand how conditions, loops, strings, variables, counters, and user input work together to build a real program.